System design (v1, MVP-focused)

Status: draft for team review. Numbers marked illustrative are assumptions to be validated by load testing, not guarantees.

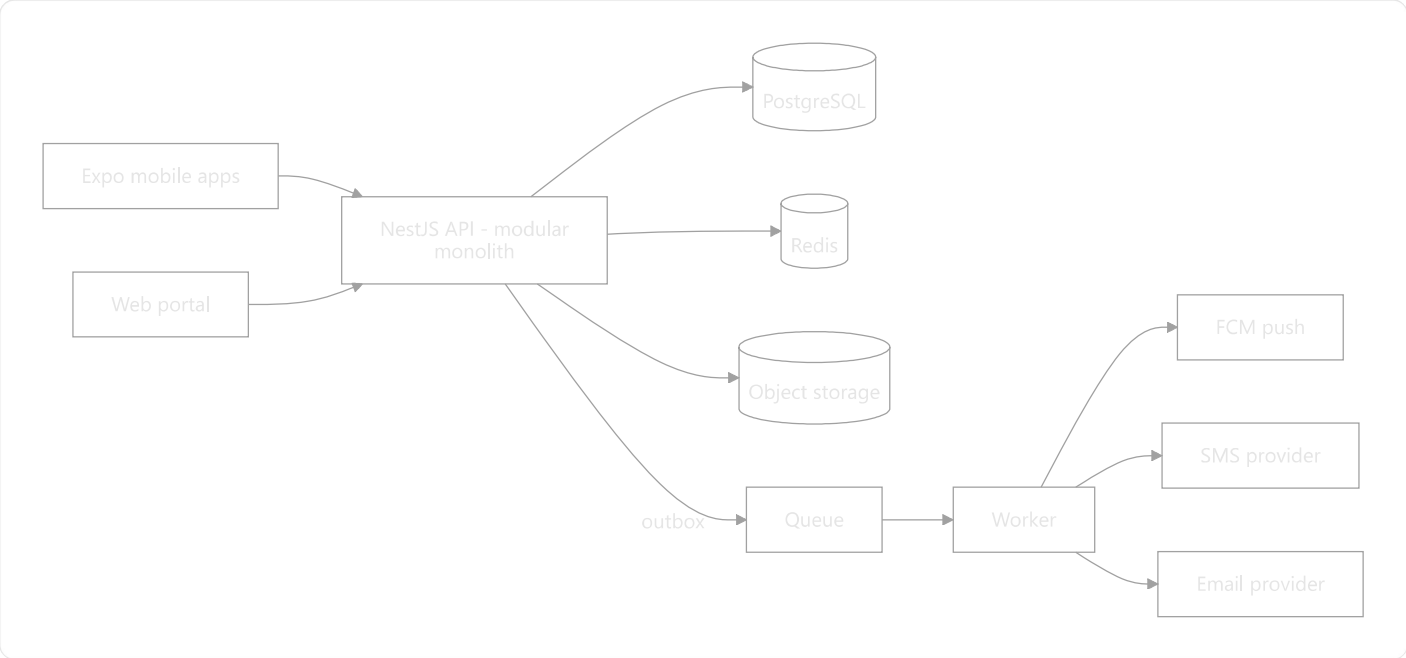
1. Goals and non-goals

Goals (MVP): principal onboarding with verification, school setup, staff and student management, parent linking, attendance, notices, homework, basic fees records, push notifications. Backend-enforced RBAC and school-level isolation.

Non-goals (MVP): timetable optimisation, report-card PDFs at scale, transport/library/hostel/payroll/inventory, online payments (design only), microservices, multi-region.

Quality targets (to validate): p95 read < 300 ms, p95 write < 500 ms at the pilot load below; zero cross-school data leaks (tested); RPO <= 15 min and RTO <= 4 h for production once backups exist.

2. Architecture



- One deployable API plus one worker (same codebase, different entrypoint).
- Modules own their tables and expose services, not shared repositories: `auth`, `tenancy`, `users`, `students`, `guardians`, `attendance`, `homework`, `notices`, `fees`, `notifications`, `files`, `audit`.
- Cross-module calls go through service interfaces and domain events (outbox). This keeps a later extraction possible without paying for it now.

3. Multi-tenancy and isolation

- **Model:** shared database, shared schema, `school_id` (UUID) on every tenant table. `schools` also carries a `shard_key` column from day one so a future move to per-cluster placement needs a lookup, not a rewrite.
- **Layer 1, API:** every route is `/v1/schools/:schoolId/...`. A guard verifies the caller has an active membership in that school and the required permission. No endpoint accepts `school_id` from the request body.
- **Layer 2, database:** PostgreSQL Row Level Security on tenant tables. Each request runs in a transaction that sets `app.school_id` with `set_config(..., true)` (transaction-local). Policy: `school_id = current_setting('app.school_id')::uuid`. The app connects as a non-owner role without `BYPASSRLS`.
- **Platform staff** (verifiers, support) use a separate role and path; every access is audit-logged.
- **Tests:** an automated suite creates two schools and asserts that every tenant endpoint returns 403/404 across schools and that RLS blocks raw queries without context.
- Pooling note: transaction-level `set_config` is safe with PgBouncer in transaction mode; session-level settings are not. Do not use `SET` without `LOCAL`.

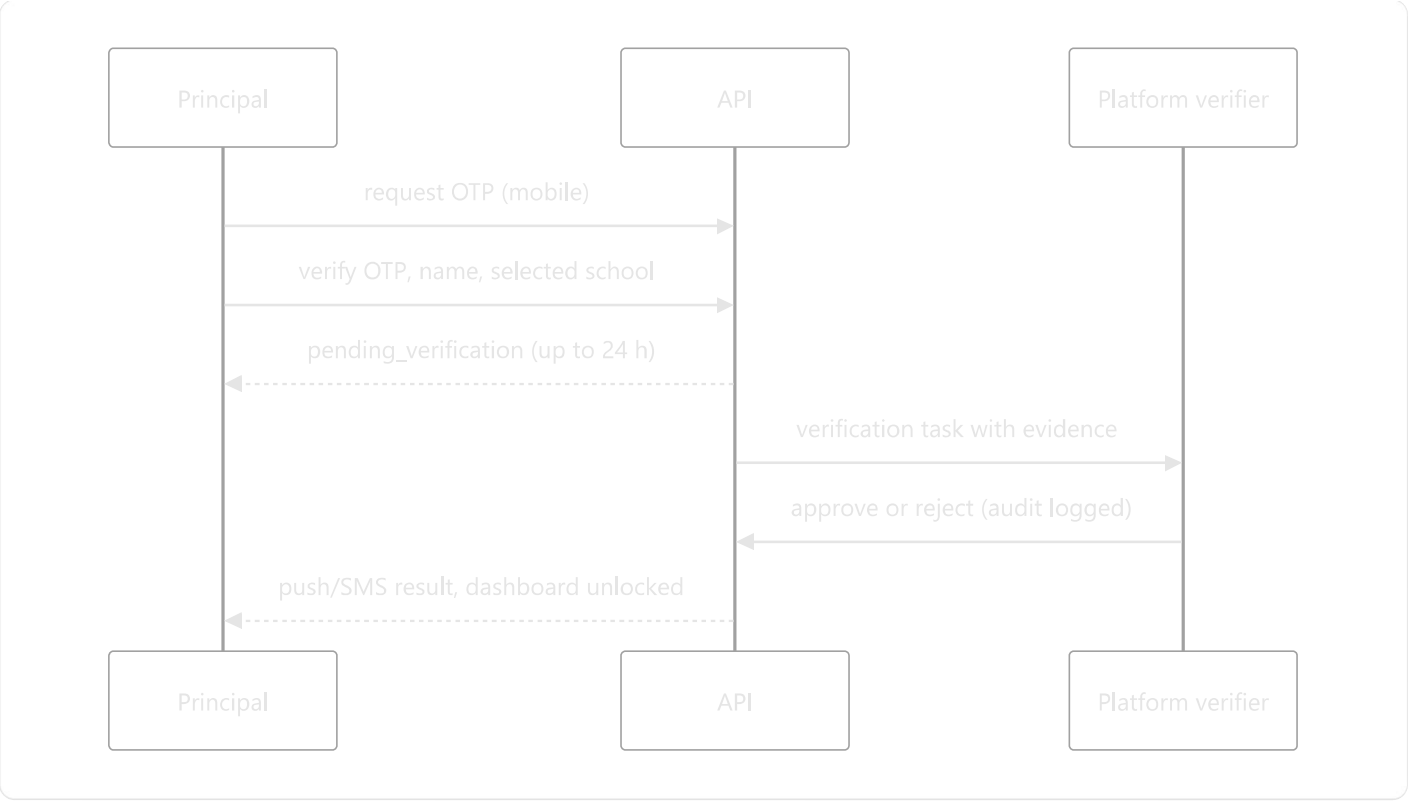
4. Identity, roles and sessions

- `users` are global (one person, one account, possibly in several schools). Access is granted by `school_memberships(user_id, school_id, role, status)`.
- Roles: principal, school_admin, vice_principal, teacher, non_teaching_staff, office_clerk, accountant, student, parent, other_staff. Permissions are a table of `role -> permission` (seeded defaults, per-school overrides later). Guards check permissions, never role names inline.
- **Login:** phone or email plus password (argon2id). OTP only for registration, password recovery and step-up (changing phone/password, fee refund, exporting data). No OTP on every login.
- **Sessions:** short-lived access JWT (15 min), rotating refresh tokens stored hashed in `sessions` with device name, last-used time, revocable per device. Refresh-token reuse detection revokes the whole chain.
- **Abuse control:** rate limits per IP, per phone and per account; OTP 6 digits, 5 min expiry, max 5 attempts, resend cooldown; generic error messages to avoid account enumeration.

5. Key flows

5.1 Principal registration and verification

States: `otp_verified -> pending_verification -> approved | rejected`. The dashboard is blocked until `approved`.

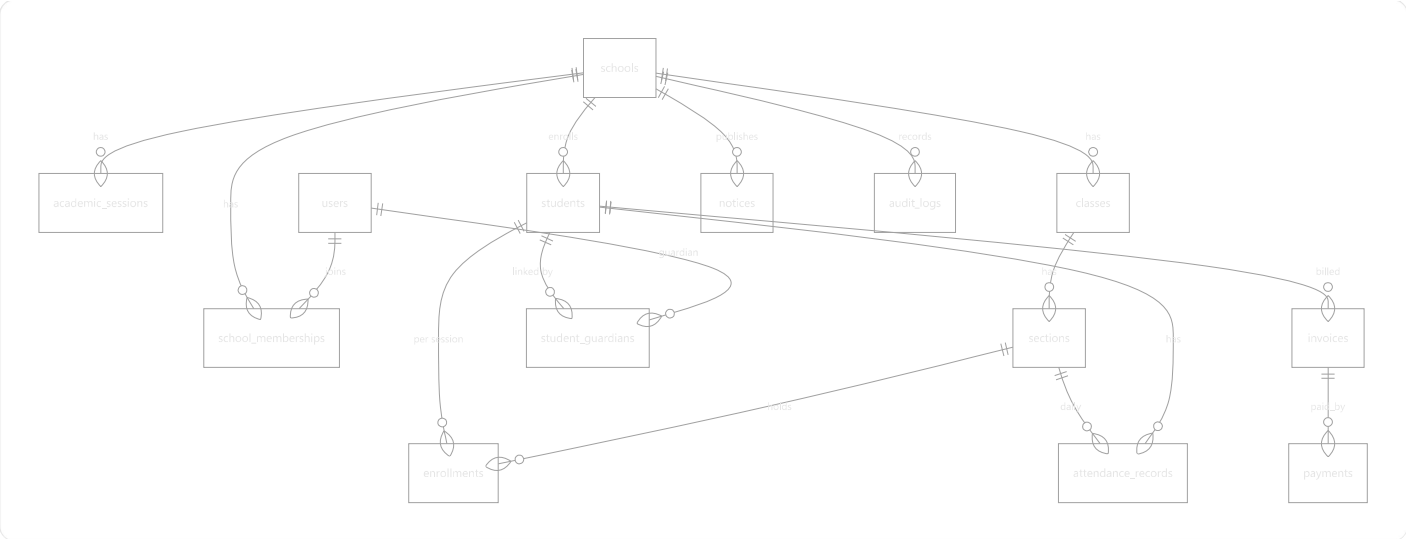


- School is selected from a `schools` directory; the typed name is never trusted. Evidence (official school ID/UDISE code, letterhead or authorization document, phone callback) is attached to a `school_claims` record. Only a platform verifier can approve.
- Two principals claiming one school: both stay pending until a verifier resolves it.
- Open: seed source for the school directory (public datasets vs manual onboarding). Availability and licence are not verified.

5.2 Parent-child linking (never by phone match)

1. School staff generate a single-use, expiring **link code** per student (stored hashed).
2. Parent enters school code, student admission number and link code.
3. API creates `student_guardians` with `status = pending` (or `active` if school policy auto-approves valid codes).
4. School may require staff approval; parent sees only `active` links.
5. Either side can revoke; everything is audit-logged. Guardian count per student is limited by school policy.

6. Data model (core)



Additional tables: `subjects`, `class_subjects`, `timetable_slots`, `homework`, `homework_submissions`, `exams`, `exam_results`, `fee_structures`, `invoice_items`, `receipts`, `notifications`, `device_tokens`, `files`, `outbox_events`, `sessions`, `otp_challenges`, `school_claims`.

Rules:

- All primary keys are UUIDv7 (time-ordered, index-friendly).
- Every tenant table: `school_id`, `created_at`, `updated_at`, `created_by`; soft delete (`deleted_at`) only where history matters.
- Student history is preserved through `enrollments` per academic session; promotion creates a new enrollment, never edits the old one. Transfer and withdrawal are statuses with dates.
- Configurable school-specific student fields: a `custom_fields` JSONB column plus a per-school field definition table; no schema change per school.
- Minimise sensitive data: do not collect what a feature does not need; encrypt sensitive columns where collected.

Attendance: one row per student per date (daily mode); unique `(school_id, student_id, date)`; monthly range partitioning from the start. Bulk submit per section is one idempotent request.

Fees (ledger style): money stored as integer paise; invoices and receipts are immutable once issued (corrections are credit notes/adjustments); payment webhooks are idempotent on gateway reference. Online payments are a later phase.

7. API design

- REST, `/v1` prefix, OpenAPI generated from NestJS; DTO validation on every input; consistent error shape with a request id.
- Cursor pagination, filterable lists, server-side sorting whitelist.
- Idempotency-Key header on attendance bulk submit, payments and any retried mobile write.
- Exports/reports are asynchronous jobs; the client polls or receives a notification and downloads via a signed URL.
- Versioning: breaking changes go to `/v2`; mobile clients send an app-version header so old versions can be told to update.

8. Async work and notifications

- **Outbox pattern:** domain events are written in the same transaction as the data change, then published to the queue by a relay. No lost or phantom notifications.
- MVP queue: BullMQ on Redis. Move to a managed queue when operational burden or durability requires it.
- Channels: push (default, cheapest), email, SMS (reserved for OTP and critical alerts because it is the largest variable cost). Per-user preferences and quiet hours.
- Jobs: notification fan-out in batches, imports/exports, report generation. All jobs are idempotent with retry and dead-letter handling.

9. Files

- Uploads go directly from the client to object storage with short-lived presigned URLs; the API stores metadata only.
- Validate type and size, store under `school_id/...` keys, private buckets, signed download URLs, malware scanning added before opening uploads to students.
- Google Drive submissions are stored as validated links only; the platform does not hold Drive credentials.

10. Caching

Start with none beyond HTTP caching. Add Redis caching for hot, read-mostly data (school settings, permissions, timetable) with explicit invalidation, and for rate limiting. Never cache across tenants without the school id in the key.

11. Security

- Backend-only authorization; no privileged keys in clients; secrets in environment-specific stores.

- TLS everywhere; encryption at rest; encrypted backups; restore tested before production data.
- `audit_logs` (append-only, separate from application tables): logins, role changes, approvals, parent links, exports, fee changes, platform-staff access.
- Dependency scanning, secret scanning, security headers, strict CORS, request size limits.
- Privacy: retention schedule per data class, deletion/anonymisation procedure, data-access log. India's DPDP Rules phase in with core obligations around May 2027; design for verifiable parental consent and breach notification now, and get legal review.

12. Observability and operations

- Structured JSON logs (request id, user id, school id, no personal data in logs), metrics (latency, error rate, queue depth, DB connections), error tracking, health and readiness endpoints, uptime check, alerting on error rate and queue lag.
- Runbooks: failed deploy rollback, OTP provider outage, database failover, data-leak incident.
- Migrations: expand-then-contract so every deploy is backward compatible and rollback-safe.

13. Capacity model (illustrative)

Pilot target: 100 schools, about 2.5 lakh registered users. At 40% monthly active, 30% of those daily, and about 5% of daily users in a peak hour: roughly 1-2k concurrent users, on the order of 100-400 requests/s at peak (result day, attendance window 8-9 am). These are planning inputs for the first load test, not predictions.

Hot spots to test first: morning attendance bulk writes, result-day reads, notification fan-out, login bursts.

14. Scaling path

Phase	Trigger (measured)	Change
0 MVP	start	One API, one Postgres, one Redis, one worker
1	CPU/connection pressure, slow reads	Connection pooler, indexes, Redis cache, read replica, horizontal API
2	Large tables, long maintenance	Partition attendance/audit/notifications by time, archive cold data, multi-AZ
3	Single-database limits	Evaluate per-cluster tenant placement using `shard_key`; extract a module only for a proven bottleneck

No claim is made that this reaches 10 crore registered users without capacity planning and load testing at each phase.

15. Build order

1. Tenancy core: schools, users, memberships, permissions, RLS, isolation test suite.
2. Auth and sessions, OTP, principal registration and verification.
3. School setup: sessions, classes, sections, subjects, staff.
4. Students, enrollments, guardian linking.
5. Attendance, notices, push notifications.
6. Homework and submissions, basic fees records.
7. Hardening: audit, load test, backup restore test, security review.

16. Open decisions

1. Database hosting (Supabase vs AWS RDS). The design uses only standard PostgreSQL features, so either works.
2. ORM (Prisma provisional) after the RLS spike.
3. School directory seed source and verifier workflow ownership.
4. Auth: own implementation vs managed provider.
5. SMS and email provider selection (DLT registration lead time in India).
6. Fees: online payment gateway choice and timing.
7. Legal review of DPDP obligations for a school-data processor.